

Facultad de Ingeniería

Programa de Ingeniería de Software

Proyecto RESUA: Asistente Multimodal de búsqueda de registros de avistamientos

Juan Diego Rojas Abarka - Manuela Velez Betancourt

Armenia, Quindío

Octubre de 2026

1. Resumen del proyecto

RESUA es una plataforma virtual en la que las personas registran avistamientos de especies silvestres en zonas urbanas de Armenia, Quindío. El proyecto desarrolla, de forma incremental dentro de un único notebook de Colab, el prototipo de un chat que permite consultar estos registros y la documentación de la plataforma usando modelos de lenguaje (LLM).

1. Fundamentos: tokens, contexto, temperatura y ejecución de un modelo local.
2. Clasificación de solicitudes con salida estructurada (JSON validado).
3. Uso seguro de una herramienta para consultar el estado de un avistamiento.
4. Búsqueda semántica con embeddings.
5. Construcción de un índice a partir de documentos.
6. Generación aumentada por recuperación (RAG) con Supabase y Groq, con respuestas citadas.
7. Evaluación y comparación de estrategias de recuperación (semántica, híbrida y con reranking).

2. Arquitectura general

El flujo completo que el notebook termina de armar es el siguiente:

1. Entrada del usuario. Se valida que sea texto de entre 1 y 2000 caracteres.
2. Clasificación. Un LLM asigna categoría, prioridad, resumen, datos faltantes, si requiere revisión humana y un nivel de confianza. La salida se valida con Pydantic.
3. Decisión. Según la clasificación, el sistema pide aclaración, deriva a una persona o continúa.
4. Herramienta (solo lectura). Si el usuario pregunta por el estado de un avistamiento con un código `AVS-0000`, el modelo propone una llamada a la función `consultar_estado_avistamiento`. El código valida la herramienta, los argumentos y que el código aparezca realmente en el texto del usuario antes de ejecutarla.
5. Consulta documental (RAG). Para preguntas sobre cómo usar RESUA, se recuperan fragmentos de documentos desde Supabase (búsqueda vectorial por coseno) y un LLM redacta una respuesta solo con esa evidencia, citando las fuentes con etiquetas `[F1]`, `[F2]`…
6. Trazabilidad. Cada ejecución registra modelo, versión del prompt, latencia, tokens y resultado, y al final se exporta la evidencia en JSON y CSV.

3. Estructura del notebook

3.1 Módulo 1: tokens, contexto e inferencia

Objetivo: entender cómo «ve» un modelo el texto y cómo afectan los parámetros de generación.

- Tokenización. Con `tiktoken` (codificación `o200k_base`) se comparan una consulta de usuario y un registro de base de datos para mostrar que el número de tokens no coincide con el número de palabras.
- Temperatura. Con logits simulados y una función softmax con temperatura se muestra cómo, a temperatura baja (0.2), el modelo concentra la probabilidad en el candidato más probable (zarigüeya, 97.1 %), mientras que a temperaturas altas reparte la probabilidad entre más opciones. Incluye una gráfica de barras.
- Prompt de enrutamiento. Se define un prompt de sistema que obliga al modelo a responder únicamente con un JSON que elige una de tres herramientas de la plataforma (`GetAllObservations`, `GetObservationsBySpecies`, `GetObservationsByUser`) o devuelve un error si la consulta no trata de observaciones de especies.
- Modelo local con Ollama. Se instala Ollama en la sesión de Colab, se descarga el modelo `gemma4:e2b` y se mide tokens de entrada/salida y latencia con temperaturas 0.0, 0.7 y 1.2.
- Decisión documentada: se conserva `temperature = 0.0`, porque la tarea exige JSON estricto y una salida determinista; las temperaturas altas aumentan el riesgo de romper la sintaxis.

3.2 Módulo 2: clasificación estructurada con Groq

Se usa la API de Groq con el modelo `openai/gpt-oss-20b`.

- **Contrato de salida.** Una clase Pydantic (`SolicitudResua`, luego `Solicitud`) define los campos `categoria`, `prioridad`, `resumen`, `datos_faltantes`, `requiere_humano` y `confianza`, con valores permitidos y límites (por ejemplo, confianza entre 0 y 1). El mismo contrato se envía a Groq como esquema JSON estricto.
- **Prompt del sistema.** Trata el texto del usuario como dato no confiable (defensa contra inyección de prompt), prohíbe inventar especies, fechas o ubicaciones y pide aclaración cuando la intención no es clara.
- **Dos versiones del prompt.** La v1 clasifica en avistamiento, identificación, soporte o general. La v2 (`resua_v2_herramienta`) añade reglas para consultar el estado de un avistamiento por código y obliga a revisión humana en acciones como modificar, eliminar, validar o aprobar registros, en pedidos de credenciales, en emergencias con fauna y en peticiones ajenas al propósito de RESUA.
- **Modo simulado (mock).** Permite probar todo el flujo sin gastar API: respuestas programadas para cada caso de prueba.
- **Casos de prueba.** Normal, ambiguo, incompleto, malicioso y fuera de alcance.
- **Regla de decisión.** `validar_y_decidir` produce `ERROR_FORMATO`, `OK_REQUIERE_HUMANO`, `OK_PIDE_ACLARACION` u `OK_VALIDADO`; la revisión humana tiene precedencia sobre la aclaración.

3.3 Módulo 3: herramienta de consulta de estado

- Una base ficticia `AVISTAMIENTOS` contiene los códigos `AVS-1001`, `AVS-1002` y `AVS-1003`.
- La función `consultar_estado_avistamiento` es de **solo lectura** y admite un parámetro para simular fallos del servicio.
- La validación de argumentos exige el patrón `^AVS-[0-9]{4}$`, prohíbe campos adicionales y comprueba que el código propuesto por el modelo exista en el texto original del usuario.

- La función `ejecutar` integra todo el recorrido (entrada → clasificación → decisión → propuesta de herramienta → validación → consulta → respuesta) y deja una traza con etapa, estado final, latencia y tokens.
- Se incluyen diez pruebas de continuidad con el estado esperado de cada una (`OK_CONSULTA`, `NO_ENCONTRADA`, `OK_REQUIERE_HUMANO`, etc.) y **pruebas negativas sin API** (herramienta inexistente, argumentos incompletos, código con formato inválido, campo extra, código que no corresponde a la entrada, fallo controlado y comprobación de que los datos no se modifican).

3.4 Módulo 4: embeddings y búsqueda semántica

- Se define un corpus pequeño de cinco pasajes sobre cómo registrar avistamientos, usar fotografías, consultar el estado e identificar especies.
- Con `sentence-transformers` y el modelo multilingüe `paraphrase-multilingual-MiniLM-L12-v2` (vectores de 384 dimensiones) se convierte el texto en vectores normalizados.
- La búsqueda calcula la similitud coseno entre la consulta y cada pasaje, y se contrasta con una búsqueda literal para mostrar la diferencia entre coincidencia por palabras y por significado.

3.5 Módulo 5: de los documentos al índice

- Se crean dos documentos Markdown en la carpeta `corpus_resua`: una guía de registro de avistamientos y una guía de seguimiento e identificación.
- Se limpian, se calcula su huella SHA-256 y se les asigna versión.
- Se comparan **dos estrategias de fragmentación** con un presupuesto de 80 tokens por fragmento y solapamiento de 12: *ventanas* de tokens y *párrafos*. Cada fragmento lleva un encabezado «título / sección» y metadatos (fuente, sección, versión, huella).
- El índice (vectores `.npy`, registros `.json` y configuración `.json`) se guarda en `indice_resua`, se recarga desde disco comprobando que las búsquedas den los mismos resultados, y se empaqueta en `RESUA_Indice_y_Corpus.zip`. Se eligió la estrategia por **párrafos**.

3.6 Módulo 6: RAG con Supabase y Groq

- **Conexión segura.** Las credenciales se leen de los secretos de Colab. Se exige una clave *publicable* de Supabase (nunca una administrativa) y se inicia sesión con un usuario de Auth, de modo que cada fila pertenece a un propietario (`owner_id`).
- **Lectura y validación del índice.** Al subir el ZIP se verifican tamaño, nombres repetidos, dimensiones (384), normalización de los vectores, metadatos obligatorios y que cada fragmento corresponda a la huella de su documento original.
- **Persistencia.** Los fragmentos y sus embeddings se guardan en la tabla `resua_fragmentos` de Supabase por lotes de 50 (operación *upsert*), bajo un identificador de corpus que incluye una huella del índice.
- **Recuperación.** La función RPC `resua_buscar_fragmentos` devuelve los *k* fragmentos más cercanos por coseno.
- **Generación con contrato.** El modelo debe devolver un JSON con `estado` (`respondida`, `parcial`, `aclaracion` o `evidencia_insuficiente`), `respuesta`, `citas` y `limitacion`. El código verifica que las citas existan en la evidencia recuperada, que coincidan con las etiquetas usadas en el texto y que las respuestas documentales tengan al menos una cita.

- **Pruebas de comportamiento** con cinco preguntas (completa, ambigua, sin evidencia, estado de un avistamiento concreto y pedido de certeza imposible).
- **Persistencia comprobada.** Un segundo cliente recupera los mismos fragmentos sin recargar el corpus, y se exporta `Clase_08_Evidencia_RAG.json`.

3.7 Módulo 7: mejora y evaluación del RAG

Se comparan tres estrategias de recuperación:

Opción	Descripción
A_semantica	Solo búsqueda vectorial en Supabase
B_hibrida_RRF	Semántica + BM25 (palabras clave) fusionadas con Reciprocal Rank Fusion
C_hibrida_reranking	Híbrida + reordenamiento con un *cross-encoder* multilingüe

Se anotan ocho casos (registro, evidencia, seguimiento, identificación y comportamiento) con la evidencia esperada y se miden Precision@3, Recall@3, Hit@3, MRR, cobertura y latencia. Las funciones de fusión y las métricas se validan con pruebas sintéticas. Hay además un grupo «reserva» de casos que no debe usarse para ajustar la decisión y que, por defecto, permanece sin abrir.

Decisión final: se elige **A_semantica** por su menor latencia y resultados competitivos; las mejoras de B y C no fueron consistentes entre grupos.

3.8 Cierre: generación final y evidencia

Se genera la respuesta RAG con la estrategia elegida, se muestra con sus fuentes, se registra una revisión manual (correcta, fiel a la evidencia, citas que respaldan, estado adecuado) y se exporta `RESUA_Evidencia_Evaluacion_RAG.zip`, que contiene un JSON con toda la configuración y los resultados y un CSV con las métricas.

4. Tecnologías utilizadas

Componente	Uso
Google Colab	Entorno de ejecución y gestión de secretos
Ollama + `gemma4:e2b`	Modelo local para las pruebas de tokens y temperatura

Componente	Uso
Groq + `openai/gpt-oss-20b`	Clasificación, propuesta de herramienta y generación RAG
Pydantic v2	Contratos y validación de salidas del modelo
`tiktoken`	Conteo de tokens de referencia
`sentence-transformers`	Embeddings multilingües (384 dim.) y reranker
`rank-bm25`	Búsqueda léxica
Supabase (Postgres + pgvector + Auth)	Almacenamiento y búsqueda vectorial de fragmentos
pandas, NumPy, matplotlib	Tablas, cálculo y gráficas

5. Decisiones de diseño y seguridad

- **El modelo propone, el código decide.** El LLM nunca ejecuta nada directamente: sus propuestas pasan por validadores estrictos.
- **Solo lectura.** La única herramienta disponible consulta estados; las acciones de modificar, aprobar o eliminar se derivan a una persona.
- **Entradas no confiables.** El texto del usuario y los documentos recuperados se tratan como datos, no como instrucciones.
- **Sin invención.** Los prompts prohíben inventar especies, fechas, estados o requisitos; si la evidencia no alcanza, el estado es `evidencia_insuficiente`.
- **Trazabilidad y reproducibilidad.** Se registran versiones de prompt, modelos con su revisión exacta, huellas SHA-256 de corpus e índice y métricas por ejecución.
- **Credenciales fuera del código.** Se usan secretos de Colab y claves publicables con control por propietario.

6. Resultados observados en la ejecución guardada

- Con el modelo local, el conteo de tokens de entrada fue 262 frente a 259 estimados con `tiktoken`. Las tres temperaturas devolvieron el mismo JSON correcto. La latencia fue alta (entre 77 y 137 segundos) porque `ollama ps` reportó ejecución en **CPU** y no en GPU.
- Con Groq, las llamadas tardaron aproximadamente entre 0.65 y 1.25 segundos y consumieron entre unos 950 y 1 180 tokens.
- En la clasificación, los casos normal, de emergencia y ambiguo se clasificaron como se esperaba, y la prueba negativa detectó correctamente categorías y prioridades inválidas (`ERROR_FORMATO`).
- Una de las ejecuciones del caso malicioso terminó en `ERROR_TECNICO`, probablemente por un rechazo de seguridad del modelo; el código contempla este escenario.

7. Limitaciones

- El corpus es didáctico y muy pequeño (dos documentos); las métricas no demuestran por sí solas la calidad en un escenario real.
- Los avistamientos de la herramienta son datos ficticios; aún no hay conexión con la base de datos real de RESUA.
- Los tiempos medidos corresponden a ejecuciones puntuales.
- El esquema de Supabase (tabla y función de búsqueda) no se crea dentro del notebook; se asume preparado de antemano.
- La revisión manual de la respuesta final está marcada como aprobada directamente en el código; conviene confirmarla leyendo la respuesta y sus fuentes.

8. Instrucciones de ejecución

8.1 Requisitos previos

1. Una cuenta de Google para usar Google Colab.
2. Una cuenta gratuita en Groq y una **clave de API**.
3. Un proyecto en Supabase con:
    - la extensión `vector` (pgvector) habilitada;
    - un **usuario de Auth** con correo y contraseña ya confirmado;
    - la **Project URL** (termina en `.supabase.co`) y la **clave publicable** (empieza por `sb_publishable_`).
4. Para el Módulo 1 (modelo local), un entorno de ejecución con **GPU** (por ejemplo, T4). Los demás módulos funcionan en CPU.

8.2 Preparar Supabase (una sola vez)

El notebook usa una tabla `resua_fragmentos` y una función `resua_buscar_fragmentos`, pero no las crea. Si todavía no existen, el siguiente SQL es una propuesta deducida de cómo las usa el código; ejecútelo en el *SQL Editor* de Supabase y ajústelo si su proyecto ya tiene un esquema propio.

```
create extension if not exists vector;

create table if not exists public.resua_fragmentos (
  id bigint generated always as identity primary key,
  owner_id uuid not null default auth.uid(),
  corpus_id text not null,
  chunk_id text not null,
  content text not null,
  metadata jsonb not null,
  configuracion jsonb not null,
  embedding vector(384) not null,
  unique (owner_id, corpus_id, chunk_id)
);

alter table public.resua_fragmentos enable row level security;

create policy resua_propietario on public.resua_fragmentos
  for all to authenticated
```

```
  using (owner_id = auth.uid())
  with check (owner_id = auth.uid());

create or replace function public.resua_buscar_fragmentos(
  query_embedding vector(384),
  p_corpus_id text,
  p_modelo text,
  p_revision text,
  match_count int default 3
)
returns table (chunk_id text, content text, metadata jsonb, similitud float)
language sql stable security invoker as $$
  select f.chunk_id, f.content, f.metadata,
         1 - (f.embedding <=> query_embedding) as similitud
  from public.resua_fragmentos f
  where f.owner_id = auth.uid()
    and f.corpus_id = p_corpus_id
    and f.configuracion->>'modelo' = p_modelo
    and f.configuracion->>'revision' = p_revision
  order by f.embedding <=> query_embedding
  limit match_count;
$$;
```

8.3 Abrir el notebook en Colab

1. Entre a Google Colab y elija **Archivo → Subir cuaderno**, luego seleccione `Proyecto.ipynb`.
2. Para el Módulo 1, elija **Entorno de ejecución → Cambiar tipo de entorno de ejecución → GPU**. Si no necesita probar el modelo local, puede omitir ese módulo y usar CPU.

8.4 Configurar los secretos

En el panel izquierdo de Colab abra el icono de la llave (**Secretos**), cree los siguientes secretos y active **Acceso desde el cuaderno** en cada uno:

Nombre del secreto	Contenido
`GROQ_API_KEY`	Clave de la API de Groq
`SUPABASE_URL`	Project URL de Supabase (https://…supabase.co)
`SUPABASE_PUBLISHABLE_KEY`	Clave publicable (`sb_publishable_…`)
`SUPABASE_EMAIL`	Correo del usuario de Auth
`SUPABASE_PASSWORD`	Contraseña del usuario de Auth

Atención: la primera celda de configuración de Groq (la que define `MODO_DEMO` y `PROMPT_VERSION`) lee un secreto llamado `proyecto`. Cree también ese secreto con la misma clave de Groq, o cambie `userdata.get("proyecto")` por `userdata.get("GROQ_API_KEY")`.

8.5 Orden de ejecución recomendado

Ejecute las celdas de arriba hacia abajo, módulo por módulo:

1. **Módulo 1 (modelo local).** Instalación de librerías, tokenización, temperatura, instalación de Ollama, descarga de `gemma4:e2b` y pruebas. Verifique que `ollama ps` indique GPU. Este módulo es opcional para el resto del proyecto.
2. **Módulo 2 y 3 (Groq).** Instale `groq` y `pydantic`. Para probar sin gastar API, ponga `MODO = "mock"`; para usar el modelo real, `MODO = "groq"`. Ejecute las celdas del contrato, prompts, base ficticia, herramienta, `ejecutar` y las pruebas.
3. **Módulo 4 y 5 (embeddings e índice).** Instale `sentence-transformers`, construya el corpus, genere los vectores y guarde el índice. Al final, la celda de descarga entrega **`RESUA_Indice_y_Corpus.zip`**: conserve ese archivo.
4. **Módulo 6 (RAG con Supabase).** Ejecute la conexión, suba el ZIP cuando la celda lo pida, elija el nombre del corpus, guarde los fragmentos en Supabase, recupere candidatos y genere respuestas. Al final se descarga **`Clase_08_Evidencia_RAG.json`**.
5. **Módulo 7 (evaluación).** Ejecute la preparación, **suba `Clase_08_Evidencia_RAG.json`** cuando se solicite, y continúe con BM25, RRF, reranking, métricas y la decisión de estrategia. Deje `ABRIR_RESERVA = False` salvo que ya haya cerrado la decisión.
6. **Cierre.** Escriba su pregunta en el campo `CONSULTA`, genere la respuesta final, revise la respuesta con sus fuentes y ejecute la última celda, que descarga **`RESUA_Evidencia_Evaluacion_RAG.zip`**.

8.6 Ajustes necesarios antes de ejecutar todo de corrido

Al revisar el código se detectaron puntos que pueden provocar errores si el notebook se ejecuta desde una sesión nueva:

- **Importaciones y constantes.** Varias celdas del Módulo 6 usan nombres que solo se importan o se definen más adelante (`userdata`, `files`, `urlparse`, `create_client`, `ClientOptions`, `BytesIO`, `perf_counter`, `Groq`). Ejecute primero la celda de preparación del Módulo 7 (la que contiene `from google.colab import userdata, files`, `from supabase import create_client, ClientOptions` y las demás importaciones), o cópiela justo después de instalar las librerías del Módulo 6. Defina además:

  ```
  MODELO_EMBEDDINGS = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
  MODELO_GENERATIVO = "openai/gpt-oss-20b"
  ```
- **Paquete `rank-bm25`.** No se instala en ninguna celda. Antes del Módulo 7 ejecute `%pip -q install rank-bm25`.
- **Función `proponer_herramienta` en modo Groq.** Su instrucción final `return { ... }, respuesta.usage` está partida en dos líneas, de modo que Python devuelve solo el diccionario y no el par (resultado, uso). Encierre ambos valores entre paréntesis, es decir `return ({...}, respuesta.usage)`, para que el modo `groq` funcione. El modo `mock` no se ve afectado.
- **Límite de la API.** Las pruebas de continuidad esperan 4 segundos entre llamadas para respetar los límites de Groq; no las acelere.

8.7 Problemas frecuentes

Síntoma	Causa probable y solución
`Revisa el secreto … y su permiso en Colab`	El secreto no existe, está vacío o no tiene activado el acceso desde el cuaderno.

Síntoma	Causa probable y solución
`Falta GROQ_API_KEY`	Falta el secreto, o la celda usa el nombre `proyecto` en lugar de `GROQ_API_KEY` (ver 8.4).
`Esta práctica requiere una clave publicable`	Se pegó una clave administrativa. Use la que empieza por `sb_publishable_`.
`No se pudo iniciar sesión`	El usuario de Auth no existe o no confirmó su correo.
`No hay fragmentos disponibles`	La tabla no se pobló, el `CORPUS_ID` no coincide, o faltan las políticas de seguridad o la función RPC en Supabase.
Error 429 de Groq	Se alcanzó el límite de uso; espere unos segundos o reduzca las llamadas.
El modelo local tarda minutos	El entorno está en CPU. Cambie a GPU y reinicie la sesión.
`ModuleNotFoundError: rank_bm25`	Instale `rank-bm25` (ver 8.6).
Los archivos descargados desaparecen	Colab borra los archivos al cerrar la sesión; conserve el ZIP del índice y el JSON de evidencia para continuar después.